

Personal Assistant AI

Guia definitivo para configurar o projeto do zero

Este documento explica, passo a passo, como montar um assistente pessoal que funciona pelo WhatsApp — organiza sua agenda, tarefas e finanças automaticamente, entendendo mensagens de texto ou áudio.

Foi escrito para quem **não tem conhecimento técnico**. Siga a ordem, sem pular etapas. Sempre que houver um bloco cinza (como o de baixo), é texto para **copiar e colar exatamente como está**.

Exemplo de bloco para copiar e colar

Todo o projeto roda em ferramentas gratuitas (dentro dos limites dos planos free do Google e da Meta) e 100% na nuvem — nada precisa ser instalado no seu computador.

Repositório oficial do código: <https://github.com/campello83/personal-assistant-ai>

Sumário

1. Visão geral e pré-requisitos
 2. Parte 1 — Criar a planilha e o projeto no Google Apps Script
 3. Parte 2 — Criar o app do WhatsApp (Meta for Developers)
 4. Parte 3 — Gerar a chave do Gemini (a “inteligência” do assistente)
 5. Parte 4 — Configurar as credenciais no Apps Script
 6. Parte 5 — Colocar o código no projeto
 7. Parte 6 — Ativar os serviços do Google (Calendar e Tasks)
 8. Parte 7 — Publicar o assistente (Web App)
 9. Parte 8 — Conectar o Webhook do WhatsApp
 10. Parte 9 — Configurar os gatilhos automáticos (lembretes e checagem de saúde)
 11. Parte 10 — Teste geral do assistente
 12. Parte 11 (opcional/avançada) — Sincronização automática com o GitHub
 13. O que o assistente sabe fazer hoje
 14. Estrutura do projeto (para referência)
 15. Problemas comuns e soluções
 16. Glossário rápido
-

1. Visão geral e pré-requisitos

O assistente recebe uma mensagem sua no WhatsApp (texto ou áudio), entende o que você quer usando o Gemini (inteligência artificial do Google) e executa a ação: marca um compromisso, cria uma tarefa, registra um gasto, gera um relatório, entre outras.

Antes de começar, você vai precisar de:

- Uma conta Google (Gmail).
- Uma conta no Facebook/Meta (para criar o app do WhatsApp).
- Um celular com WhatsApp para os testes (a Meta fornece um número de teste próprio, você não precisa usar seu número pessoal).
- Uma conta no GitHub (gratuita, em github.com) — é onde o código-fonte do projeto fica guardado.
- Opcional: uma conta no Postman, útil só para diagnosticar problemas mais técnicos.

Tempo estimado: entre 1h30 e 2h30 para a configuração completa, na primeira vez.

2. Parte 1 — Criar a planilha e o projeto no Google Apps Script

A planilha do Google Sheets funciona como o “banco de dados” do assistente: guarda o histórico de mensagens, os gastos, os logs e a memória de curto prazo.

1. Acesse sheets.google.com e crie uma **Planilha em branco**.
2. Renomeie a planilha (clique no título, no canto superior esquerdo) para:

Personal Assistant AI - DB

3. Renomeie a primeira aba (clique com o botão direito na aba “Página1”, no rodapé) para:

Logs

4. Na linha 1 dessa aba, preencha as células A1 a E1 com:

Timestamp | Tipo | Origem | Mensagem | Status

5. No menu superior, clique em **Extensões → Apps Script**. Isso abre o editor de código, já conectado a essa planilha.
6. No editor, clique no título “Projeto sem título” (canto superior esquerdo) e renomeie para:

Personal Assistant AI

Pronto — você tem a “casa” onde o código do assistente vai morar. As próximas partes vão preencher esse projeto.

3. Parte 2 — Criar o app do WhatsApp (Meta for Developers)

Aqui você cria a conexão entre o WhatsApp e o seu assistente.

3.1 — Criar o app

1. Acesse developers.facebook.com e clique em “**Meus Apps**”.
2. Clique em “**Criar App**”.
3. Na tela de “Casos de uso”, escolha a opção com o ícone do WhatsApp (algo como “**Conectar-se com os clientes pelo WhatsApp**”) e avance.
4. Se for pedido um **Portfólio Comercial**: crie um novo, com nome do portfólio, seu nome e e-mail comercial. Confirme o e-mail se solicitado.
5. Revise a tela de requisitos e avance.
6. Na tela final, confirme e clique em “**Criar aplicativo**” (pode pedir sua senha novamente).

3.2 — Anotar as credenciais iniciais

1. No painel do app, vá em **Casos de uso → Conectar-se com os clientes pelo WhatsApp**.
2. Na tela “**Configuração da API**”, anote em algum lugar seguro (não compartilhe com ninguém):
 - O **Token de acesso temporário** (dura 24h — mais adiante trocamos por um permanente).
 - O **Phone Number ID**.
 - O **número de telefone de teste** fornecido pela Meta.

⚠ **Atenção — restrição de país:** o número de teste padrão da Meta é registrado nos EUA. Desde setembro/2025, mensagens enviadas pela API a partir dos EUA para números do Brasil ou Indonésia podem ser bloqueadas. Isso **não** afeta você mandar mensagens para o número de teste — só afetaria o assistente mandando mensagens automáticas para números BR/ID quando o app estiver em produção com um número próprio.

4. Parte 3 — Gerar a chave do Gemini (a “inteligência” do

assistente)

O Gemini é quem lê sua mensagem e entende o que você quer fazer.

1. Acesse aistudio.google.com e faça login com sua conta Google.
2. No menu esquerdo, clique em “Get API key” → “Create API key” → “Create API key in new project”.
3. Copie a chave gerada e guarde num lugar seguro.

5. Parte 4 — Configurar as credenciais no Apps Script

Volte ao editor do Apps Script (Parte 1, passo 5).

1. Clique no ícone de engrenagem ⚙️ “Configurações do projeto”, no menu lateral esquerdo.
2. Role até “Propriedades do script” → “Adicionar propriedade do script”.
3. Cadastre uma propriedade por vez, com estes nomes exatos (a coluna da esquerda é o nome, a da direita é o valor que você anotou antes):

Nome da propriedade	Valor
GEMINI_API_KEY	a chave gerada na Parte 3
WHATSAPP_TOKEN	o token de acesso da Parte 2.2
WHATSAPP_PHONE_NUMBER_ID	o Phone Number ID da Parte 2.2

4. Clique em **Salvar**.

⚙️ Mais adiante (Parte 8) o próprio código cria automaticamente outras propriedades sozinho (como o ID da lista de tarefas) — você não precisa se preocupar com elas agora.

6. Parte 5 — Colocar o código no projeto

O código-fonte completo e sempre atualizado do assistente está no repositório do GitHub:

<https://github.com/campello83/personal-assistant-ai>

Cada arquivo `.gs` do repositório corresponde a um arquivo dentro do projeto do Apps Script. Para colocar o código no seu projeto:

1. No repositório do GitHub, abra cada arquivo `.gs` (ex: `Code.gs`, `Logs.gs`, `Agenda.gs` etc.) e copie o conteúdo inteiro.
2. No editor do Apps Script, clique no “+” ao lado de “Arquivos” → **Script** → dê o mesmo nome do arquivo (sem a extensão `.gs`, o Apps Script adiciona sozinho) → cole o conteúdo copiado.
3. Repita para todos os arquivos `.gs` do repositório. Para o arquivo `Painel.html` (usado no painel visual), o processo é o mesmo, mas escolhendo **HTML** em vez de **Script** ao criar.
4. Salve o projeto (ícone de disquete ou `Ctrl+S`).

Arquivos que você vai encontrar no repositório e o que cada um faz:

Arquivo	Função
<code>Code.gs</code>	Ponto de entrada (webhook do WhatsApp) e roteador principal
<code>Gemini.gs</code>	Interpretação de mensagens (texto/áudio) usando o Gemini
<code>WhatsApp.gs</code>	Envio e recebimento de mensagens/mídia via WhatsApp
<code>Agenda.gs</code>	Criação/edição/cancelamento de compromissos no Google Calendar

Tasks.gs	Criação de tarefas no Google Tasks
Financeiro.gs	Registro de gastos, entradas e controle financeiro
Relatorios.gs	Geração de relatórios diários, semanais e mensais
Lembretes.gs	Lembretes e cobranças automáticas
Dashboard.gs / Painel.html	Painel visual (Web App) com indicadores
Config.gs	Preferências gerais e memória de curto prazo do assistente
logs.gs	Registro completo de cada movimentação do sistema
HealthCheck.gs	Checagem periódica de saúde das integrações
appsscript.json	Arquivo de configuração do projeto (manifesto)

💡 **Alternativa mais rápida (recomendada):** em vez de copiar e colar arquivo por arquivo, existe uma extensão gratuita de navegador que sincroniza o repositório inteiro direto no editor do Apps Script em poucos cliques. Veja a **Parte 11** deste guia.

7. Parte 6 — Ativar os serviços do Google (Calendar e Tasks)

O projeto usa dois “serviços avançados” do Google que precisam ser ativados manualmente uma única vez.

1. No editor do Apps Script, menu lateral → **“Serviços”** → clique no **“+”**.
2. Localize **“Google Calendar API”** → **Adicionar**.
3. Repita o processo para **“Tasks API”** → **Adicionar**.

Autorizar as permissões

Na primeira vez que o projeto tentar usar essas APIs (ou enviar mensagens pela internet), o Google vai pedir autorização. Para autorizar tudo de uma vez, sem esperar isso acontecer durante o uso real:

1. No editor, crie uma função temporária, por exemplo:

```
function autorizarTudo() {  
  UrlFetchApp.fetch("https://www.google.com");  
  CalendarApp.getDefaultCalendar();  
  Tasks.Tasklists.list();  
}
```

2. Selecione essa função no menu suspenso ao lado do botão “Executar” e clique em **Executar**.
3. Vai aparecer uma tela pedindo permissão — clique em **“Revisar permissões”**, escolha sua conta, clique em **“Avançado”** → **“Acessar Personal Assistant AI (não seguro)”** → **Permitir**.
4. Depois que autorizar, pode apagar essa função temporária.

⚠ **Importante para quem usa o deploy automático (Parte 11):** se um serviço avançado for ativado só pela interface (como acima) e essa ativação não estiver registrada no arquivo `appsscript.json` que fica no GitHub, um deploy automático futuro pode **remover essa ativação silenciosamente** — o sintoma é um erro do tipo `ReferenceError: Calendar is not defined`, mesmo já tendo funcionado antes. A solução é sempre manter o `appsscript.json` do GitHub atualizado com o bloco `enabledAdvancedServices`.

8. Parte 7 — Publicar o assistente (Web App)

Para o WhatsApp conseguir enviar mensagens até o seu projeto, ele precisa estar publicado como um endereço (URL) acessível pela internet.

1. No editor do Apps Script, clique em **Implantar** → **Nova implantação**.

2. Clique no ícone de engrenagem ⚙, ao lado de “Selecionar tipo”, e escolha **“App da Web”**.
3. Preencha:
 - **Descrição:** Webhook v1 - recepcao de mensagens do WhatsApp
 - **Executar como:** Eu
 - **Quem pode acessar:** Qualquer pessoa
4. Clique em **Implantar**.
5. Se aparecer uma tela de autorização, repita o processo da Parte 6 (Revisar permissões → Avançado → Acessar → Permitir).
6. Copie a **URL do app da Web** — você vai precisar dela na próxima parte.

⚠ Sempre que o código de qualquer arquivo mudar, é necessário publicar uma **nova versão** (não uma implantação nova): **Implantar → Gerenciar implantações → ícone de lápis (editar) → “Nova versão”** → descrever a mudança → **Implantar**. A URL do Web App não muda nunca — só o código por trás dela é atualizado. Esse é o único passo do projeto que continua manual (limitação do próprio Google, sem forma de automatizar).

9. Parte 8 — Conectar o Webhook do WhatsApp

Agora vamos avisar a Meta para onde ela deve mandar as mensagens recebidas.

1. No painel do app (Meta for Developers), vá em **WhatsApp → Configuração**, seção **Webhook → Editar**.
2. Preencha:
 - **URL de callback:** a URL copiada na Parte 7.
 - **Token de verificação:**

personalassistentai2026

3. Clique em **“Verificar e salvar”**.
4. No campo **“messages”**, clique em **“Assinar”**.

⚠ **Problema comum:** mesmo com tudo assinado, as mensagens podem não chegar se o app não estiver “inscrito” (subscribed) na Conta do WhatsApp Business (WABA). Veja a seção **15. Problemas comuns** para o passo a passo de correção via Postman.

Teste rápido

Envie uma mensagem de teste do seu WhatsApp pessoal para o número de teste fornecido pela Meta. Depois, abra a planilha e confira se apareceu uma linha nova na aba **Logs**. Se apareceu, a comunicação básica está funcionando.

10. Parte 9 — Configurar os gatilhos automáticos

O assistente tem duas rotinas que rodam sozinhas, sem você precisar mandar mensagem nenhuma: os **lembretes** (agenda, tarefas e relatórios automáticos) e a **checagem de saúde** do sistema.

10.1 — Gatilho de lembretes (a cada 15 minutos)

1. No editor do Apps Script, clique no ícone de relógio ⌚ **“Gatilhos”**, no menu lateral.
2. Clique em **+ Adicionar gatilho**.
3. Preencha:
 - **Função a ser executada:** verificarLembretes
 - **Fonte do evento:** Baseado em tempo
 - **Tipo de gatilho de tempo:** Timer de minutos
 - **Intervalo:** A cada 15 minutos
4. **Salvar** (autorize permissões se for solicitado).

10.2 — Gatilho de checagem de saúde (1x por dia)

1. Ainda na tela de Gatilhos, clique em “+ **Adicionar gatilho**” novamente.
2. Preencha:
 - **Função a ser executada:** `verificarSaudeSistema`
 - **Fonte do evento:** `Baseado em tempo`
 - **Tipo de gatilho de tempo:** `Timer diário`
 - **Horário:** uma janela tranquila, por exemplo `7h às 8h da manhã`.
 - **Escolha qual implantação deve ser executada:** `Cabeçalho` (a versão publicada, não “Teste”).
3. **Salvar.**

Se algo parar de funcionar (token expirado, cota estourada, serviço fora do ar), você recebe um aviso automático no seu WhatsApp.

11. Parte 10 — Teste geral do assistente

Com tudo publicado e configurado, envie estas mensagens de teste pelo WhatsApp (para o número de teste da Meta) e confira o resultado esperado:

Mensagem de exemplo	O que deve acontecer
“Marca uma reunião com o cliente amanhã às 15h”	Confirmação no WhatsApp + evento aparece no Google Calendar
“muda essa reunião pra 16h”	Reconhece a última reunião mencionada e altera o horário
“cancela essa reunião”	Remove o evento do Google Calendar
“Cria uma tarefa para revisar o contrato até sexta-feira”	Confirmação no WhatsApp + tarefa aparece em <code>tasks.google.com</code>
“Gastei 50 reais no mercado”	Confirmação + linha nova na aba Financeiro da planilha
“Como está meu saldo essa semana?”	Resumo financeiro do período
“Me dá um relatório do dia”	Resumo de agenda + tarefas + financeiro do dia
Abrir <code>SUA_URL_DO_WEB_APP?painel=1</code> no navegador	Painel visual com indicadores e gráfico de gastos

Se todos os itens funcionarem, o assistente está pronto para uso.

12. Parte 11 (opcional/avançada) — Sincronização automática com o GitHub

Esta parte é opcional — o assistente funciona perfeitamente sem ela. Ela existe para facilitar a manutenção do código: em vez de copiar e colar arquivo por arquivo (Parte 5), o código é sincronizado automaticamente entre o GitHub e o Apps Script.

12.1 — Instalar a extensão de sincronização

1. Na Chrome Web Store, busque por “**Google Apps Script GitHub Assistant**” e clique em “**Usar no Chrome**”.
2. Abra (ou recarregue) o editor do seu projeto no Apps Script — novos botões (**Repository**, **Branch**, **Push**, **Pull**) aparecem na barra de ferramentas.
3. Acesse `https://script.google.com/home/usersettings` e ative a opção “**Google Apps Script API**”.

12.2 — Gerar um token de acesso do GitHub

1. No GitHub: foto de perfil → **Settings** → **Developer settings** → **Personal access tokens** → **Tokens (classic)** → **Generate new token (classic)**.
2. Nome (Note): `Apps Script Extensao`. Validade: `90 days` ou `No expiration`.
3. Marque os escopos `repo` e `gist`. Se for versionar também os workflows do GitHub Actions (parte 12.4), marque também `workflow`.

4. Gere e **copie o token imediatamente** (ele só aparece uma vez).

12.3 — Conectar e vincular o repositório

1. No editor do Apps Script, clique em **“Login to SCM”**, informe seu usuário do GitHub e cole o token gerado.
2. Clique em **Repository** → selecione `personal-assistant-ai`.
3. Clique em **Branch** → selecione `main`.

A partir daqui, sempre que alterar um arquivo `.gs`, use a seta **Push ↑** para enviar ao GitHub.

12.4 — Deploy 100% automático (avançado)

Para eliminar até o clique manual no Push, é possível configurar um pipeline (GitHub Actions) que aplica automaticamente no Apps Script qualquer mudança enviada ao GitHub. Isso exige criar um workflow (`.github/workflows/deploy.yml`) usando a ferramenta `clasp` da própria Google, e cadastrar credenciais como `secrets` do repositório. É um processo mais técnico — os detalhes completos, incluindo os problemas mais comuns encontrados nesse processo, estão documentados no arquivo `GUIA-REPLICACAO.md` do repositório do projeto.

🔔 Mesmo com o deploy 100% automático, publicar uma **“Nova versão”** da implantação (Parte 7) continua sendo manual — é uma limitação do próprio Google.

13. O que o assistente sabe fazer hoje

- 📅 Agendar, alterar e cancelar compromissos no Google Calendar (inclusive por referência indireta, como “essa reunião”).
- ✔ Criar tarefas no Google Tasks, com ou sem horário.
- 📊 Registrar gastos e entradas, calcular saldo por período.
- 📄 Gerar relatórios (diário, semanal, mensal) sob demanda ou automaticamente.
- ⌚ Enviar lembretes automáticos de compromissos (1h antes, com link do Google Meet quando for reunião online) e de tarefas.
- 📈 Mostrar um painel visual com os indicadores principais.
- 🧠 Lembrar do último compromisso mencionado, para você não precisar repetir informação.
- ⚠ Avisar automaticamente se alguma parte do sistema parar de funcionar.

14. Estrutura do projeto (para referência)

```
WhatsApp (Meta Cloud API)
  |
  v
Google Apps Script (Webhook / doPost)
  |
  v
  Gemini API --> Interpreta a mensagem e devolve um JSON
  |
  v
  Roteador (Code.gs) decide o que fazer com o JSON
  |
  +-----+-----+-----+-----+
  |         |         |         |         |
  v         v         v         v         v
Agenda.gs  Tasks.gs  Financeiro.gs  Relatorios.gs  Lembretes.gs
  |         |         |         |         |
  +-----+-----+-----+-----+
  |
  v
  Google Sheets (banco de dados)
  |
  v
  Logs.gs (registro de tudo)
  HealthCheck.gs (checagem de saúde)
```

Cada assunto (agenda, tarefas, financeiro, relatórios, lembretes) tem seu próprio arquivo de código isolado. O `Code.gs` só decide para qual arquivo mandar cada pedido. Não existe banco de dados externo nem servidor próprio — é só Google Apps Script + Google Sheets.

15. Problemas comuns e soluções

App não recebe mensagens, mesmo com o Webhook configurado Confirme se o app está “inscrito” (subscribed) na Conta do WhatsApp Business. No Postman: GET `https://graph.facebook.com/v20.0/{WABA_ID}/subscribed_apps`, com o token no cabeçalho `Authorization: Bearer SEU_WHATSAPP_TOKEN`. Se vier vazio, rode a mesma URL com POST para inscrever o app. Para descobrir o WABA ID: GET `.../{PHONE_NUMBER_ID}?fields=whatsapp_business_account`.

Erro de autenticação ao enviar mensagem (códigos 131005 ou 190) O token de acesso temporário da Meta expira em 24h. Gere um novo em “Configuração da API” e atualize a propriedade `WHATSAPP_TOKEN` no Apps Script.

Erro ReferenceError: Calendar is not defined (ou Tasks is not defined) O serviço avançado correspondente (Calendar API ou Tasks API) não está ativo no projeto — normalmente acontece depois de um deploy automático que sobrescreveu o `appsscript.json`. Reative pelo menu “Serviços” (Parte 6) e, se usar o deploy automático, garanta que o `appsscript.json` no GitHub inclui o bloco `enabledAdvancedServices`.

Modelo do Gemini parou de funcionar (404 - model is no longer available) O Google descontinua modelos periodicamente. Troque o nome do modelo usado na chamada da API (dentro de `Gemini.gs`) pelo modelo atual recomendado, consultando `ai.google.dev`.

Restrição de país ao enviar mensagens para o Brasil Números de teste da Meta registrados nos EUA podem ser bloqueados de enviar mensagens via API para números do Brasil/Indonésia. Isso não afeta o recebimento de mensagens enviadas por você. Para produção, é necessário registrar um número de telefone próprio.

Datas ou horários errados em tarefas O Google Tasks não suporta horário nativamente, e o Google Sheets converte automaticamente texto parecido com data (`"2026-08-03"`) ou hora (`"00:35"`) em um objeto de data, o que pode corromper o cálculo. O código já contorna isso guardando esses valores como texto puro numa aba própria — se for alterar esse trecho do código, mantenha esse cuidado.

O assistente não reconhece “essa reunião” logo depois de criar o compromisso Verifique se o número de telefone está sendo salvo como **texto** (não número) na aba “Memoria_Contexto” da planilha — é a mesma armadilha de conversão automática do Sheets mencionada acima, já contornada no código atual.

16. Glossário rápido

Termo	O que significa
Webhook	Um endereço (URL) que fica “esperando” a Meta mandar as mensagens recebidas no WhatsApp.
Apps Script	Ferramenta gratuita do Google para rodar código automaticamente, sem precisar de um servidor próprio.
Gatilho (Trigger)	Uma regra que faz uma função do código rodar sozinha, em um horário ou intervalo definido.
Intent	A “intenção” identificada pelo Gemini a partir da sua mensagem (ex: agendar um compromisso, criar uma tarefa).
Propriedade do script	Um par de chave/valor guardado com segurança dentro do projeto do Apps Script (ex: chaves de API).
Deploy / Implantação	O processo de publicar (ou atualizar) o código para que ele funcione de verdade, com um endereço acessível pela internet.

Documento gerado como parte da Etapa 11 (documento final) do projeto Personal Assistant AI. Para o histórico completo de decisões técnicas e o passo a passo etapa por etapa (como o projeto foi construído), consulte `GUIA-REPLICACAO.md` e `CHANGELOG.md` no repositório do GitHub.